In [ ]:
```
import marimo as mo
```

Trabalho Prático 2 — Exercício 3

Descrição do Problema

O problema descrito no documento sobre a multiplicação de inteiros, tem como objetivo modelar um programa imperativo que calcula o produto de inteiros. Para isso, utilizamos um autómato de fluxo de controlo (Control Flow Automaton) como modelo.

In [ ]:
```
from z3 import Int, BitVec, BitVecVal, And, Or, Not, LShR, ULT, Solver,
sat, Implies, ZeroExt

N = 8

INIT = 0
SKIP = 1
LEFT = 2
RIGHT = 3
STOP = 4
ERROR = 5
```

Alínea a) Modelação do SFOTS

Enunciado:

> Construir um SFOTS, usando BitVec's de tamanho n, que descreva o comportamento deste autómato; para isso identifique e codifique em Z3, as variáveis do modelo, o estado inicial, a relação de transição e o estado de erro.

O que fizemos aqui: Criamos as variáveis do sistema usando vetores de bits de tamanho N, mapeámos os diferentes nodos do programa e escrevemos a lógica de transição que simula os cálculos passo a passo, tendo o cuidado com situações de overflow.

In [ ]:
```
def declare_state(suffix):
    state = {}
    state['pc'] = Int(f'pc_{suffix}')
    state['a'] = BitVec(f'a_{suffix}', N)
    state['b'] = BitVec(f'b_{suffix}', N)
    state['x'] = BitVec(f'x_{suffix}', N)
    state['y'] = BitVec(f'y_{suffix}', N)
    state['z'] = BitVec(f'z_{suffix}', N)
    return state

def init(state):
```

```
    return And(
        state['pc'] == INIT,
        state['x']  == state['a'],
        state['y']  == state['b'],
        state['z']  == BitVecVal(0, N)
    )

def error(state):
    return state['pc'] == ERROR

def trans(curr, next_st):
    x_overflow = LShR(curr['x'], N - 1) == BitVecVal(1, N)
    z_overflow = ULT(curr['z'] + curr['x'], curr['z'])

    t_init = And(curr['pc'] == INIT, next_st['pc'] == SKIP,
                 next_st['a'] == curr['a'], next_st['b'] == curr['b'],
                 next_st['x'] == curr['x'], next_st['y'] == curr['y'],
next_st['z'] == curr['z'])

    t_skip_to_stop = And(curr['pc'] == SKIP, curr['y'] == BitVecVal(0, N),
next_st['pc'] == STOP,
                         next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                         next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])


    t_skip_to_left = And(curr['pc'] == SKIP, curr['y'] != BitVecVal(0, N),
                         (curr['y'] & BitVecVal(1, N)) == BitVecVal(0, N),
next_st['pc'] == LEFT,
                         next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                         next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])

    t_skip_to_right = And(curr['pc'] == SKIP, curr['y'] != BitVecVal(0, N),
                          (curr['y'] & BitVecVal(1, N)) == BitVecVal(1, N),
next_st['pc'] == RIGHT,
                          next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                          next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])


    t_left_to_error = And(curr['pc'] == LEFT, x_overflow, next_st['pc'] ==
ERROR,
                          next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                          next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])

    t_left_to_skip = And(curr['pc'] == LEFT, Not(x_overflow), next_st['pc']
== SKIP,
                         next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                         next_st['x'] == curr['x'] << 1, next_st['y'] ==
```

```
LShR(curr['y'], 1), next_st['z'] == curr['z'])

    t_right_to_error = And(curr['pc'] == RIGHT, z_overflow, next_st['pc']
== ERROR,
                           next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                           next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])

    t_right_to_skip = And(curr['pc'] == RIGHT, Not(z_overflow),
next_st['pc'] == SKIP,
                          next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                          next_st['x'] == curr['x'], next_st['y'] ==
curr['y'] - BitVecVal(1, N),
                          next_st['z'] == curr['z'] + curr['x'])

    t_stop_loop = And(curr['pc'] == STOP, next_st['pc'] == STOP,
                      next_st['a'] == curr['a'], next_st['b'] == curr['b'],
                      next_st['x'] == curr['x'], next_st['y'] == curr['y'],
next_st['z'] == curr['z'])

    t_error_loop = And(curr['pc'] == ERROR, next_st['pc'] == ERROR,
                       next_st['a'] == curr['a'], next_st['b'] ==
curr['b'],
                       next_st['x'] == curr['x'], next_st['y'] ==
curr['y'], next_st['z'] == curr['z'])


    return Or(t_init, t_skip_to_stop, t_skip_to_left, t_skip_to_right,
              t_left_to_error, t_left_to_skip, t_right_to_error,
t_right_to_skip,
              t_stop_loop, t_error_loop)
```

Alínea b) Verificação do Invariante

Enunciado:

> Usando k-indução verifique nesse SFOTS se a propriedade $(x * y + z = a * b)$ é um invariante do seu comportamento.

O que fizemos neste bloco: Definimos o invariante $x \cdot y + z = a \cdot b$ e aplicámos um método de k-indução para garantir que esta igualdade nunca se perde durante a execução do programa.

In [ ]:
```
def invariant(state):
    x = ZeroExt(N, state['x'])
    y = ZeroExt(N, state['y'])
    z = ZeroExt(N, state['z'])
    a = ZeroExt(N, state['a'])
    b = ZeroExt(N, state['b'])
```

```
    return Implies(
        state['pc'] != ERROR,
        x * y + z == a * b
    )

def k_induction_invariant(declare_state, init, trans, inv, k, N):
    print(f"A correr k = {k}")
    # Caso base
    s_base = Solver()
    s_base.set("timeout", 15000)
    states = [declare_state(i) for i in range(k)]
    constraints = []

    constraints.append(init(states[0]))

    for i in range(k - 1):
        constraints.append(trans(states[i], states[i+1]))

    constraints.append(Or([Not(inv(st)) for st in states]))

    s_base.add(And(constraints))

    if s_base.check() == sat:
        print(f"O caso base falhou")
        return False
    else:
        print("Base OK")

    # Indutivo
    s_step = Solver()
    s_step.set("timeout", 15000)
    states = [declare_state(i) for i in range(k + 1)]
    constraints = []


    for i in range(k):
        constraints.append(inv(states[i]))

    for i in range(k):
        constraints.append(trans(states[i], states[i+1]))

    constraints.append(Not(inv(states[k])))

    s_step.add(And(constraints))

    if s_step.check() == sat:
        print(f"O passo indutivo falhou para K = {k}")
        return False

    else:
        print(f"Indução OK (k={k})")
        return True

k_induction_invariant(declare_state, init, trans, invariant, k=2, N=8)
```

```
A correr k = 2
Base OK
Indução OK (k=2)
```
True

Alínea c) Prova de Segurança com Pré-condições

Enunciado:

> Usando k-indução no SFOTS acima e adicionando ao estado inicial a condição $(a < 2^{n/2}) \wedge (b < 2^{n/2})$, verifique a segurança do programa; nomeadamente prove que, com tal estado inicial, o estado de erro nunca é acessível.

O que fizemos aqui: Neste ponto, quisemos testar a segurança do programa. Ao impor que os números de entrada a e b não ultrapassam metade da capacidade dos bits (n/2), utilizámos novamente a k-indução para provar que, com estas condições iniciais, o programa é totalmente seguro e o erro de overflow torna-se impossível de alcançar.

In [ ]:
```python
def init_c(state):
    limite = 1 << (N // 2)

    return And(
        state['pc'] == INIT,
        state['z'] == BitVecVal(0, N),
        state['x'] == state['a'],
        state['y'] == state['b'],
        ULT(state['a'], BitVecVal(limite, N)),
        ULT(state['b'], BitVecVal(limite, N))
    )

def safety_invariant(state):
    limite = BitVecVal(1 << (N // 2), N)
    return And(
        state['pc'] != ERROR,
        invariant(state),
        ULT(state['a'], limite),
        ULT(state['b'], limite)
    )

def verificar_seguranca_c(declare_state, trans, k):
    return k_induction_invariant(declare_state, init_c, trans,
safety_invariant, k, N)

verificar_seguranca_c(declare_state, trans, k=2)
```

```
A correr k = 2
Base OK
Indução OK (k=2)
```
True

Alínea d) Propriedade de Terminação

Enunciado:

> Codifique a propriedade de animação que exprime que o programa termina quaisquer que sejam os valores iniciais de a e b e prove que ela se verifica no SFOTS que construiu.

O que fizemos aqui: Por fim, faltava garantir que o programa não fica preso em ciclos infinitos. Definimos que a condição de sucesso é atingir o nodo de paragem (STOP) e utilizámos uma verificação limitada a K passos para confirmar que o algoritmo termina sempre, independentemente dos valores iniciais escolhidos para a e b.

In [ ]:
```
def termination_property(state):
    return state['pc'] == STOP

def verificar_terminacao_d(declare_state, init_func, trans_func, prop_func,
limite, N):

    for k in range(1, limite + 1):
            s = Solver()
            states = [declare_state(i) for i in range(k)]
            constraints = []

            constraints.append(init_func(states[0]))

            for i in range(k - 1):
                constraints.append(trans_func(states[i], states[i+1]))

            for st in states:
                constraints.append(Not(prop_func(st)))

            loop_constraints = []
            for j in range(k):
                loop_constraints.append(trans_func(states[-1], states[j]))

            constraints.append(Or(loop_constraints))

            s.add(And(constraints))

            if s.check() == sat:
                print(f"Loop infinito detetado para k={k}")
                model = s.get_model()
                for i, st in enumerate(states):
                    pc_val = model.evaluate(st['pc'])
                    a_val  = model.evaluate(st['a'])
                    b_val  = model.evaluate(st['b'])
                    x_val  = model.evaluate(st['x'])
                    y_val  = model.evaluate(st['y'])
                    z_val  = model.evaluate(st['z'])
                    print(f"  Estado {i}: pc = {pc_val}, a = {a_val}, b =
```

```
{b_val}, x = {x_val}, y = {y_val}, z = {z_val}")
                return False
            else:
                print(f"sem loops, tudo limpo")

    print("A propriedade de terminação foi provada para todos os passos até
ao limite K")
    return True

verificar_terminacao_d(declare_state, init_c, trans, termination_property,
limite=15, N=N)
```

```
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
sem loops, tudo limpo
 A propriedade de terminação foi provada para todos os passos até ao limite K
```

True

Ferramentas LLM

link para o chat.